

OPEN PORT SCANNER

Project Documentation & Technical Report

Project Name:	Open Port Scanner
Team:	FOUR CHAMPS
Language:	Python 3
Date:	June 23, 2026
Version:	1.0

This document provides a complete overview of the Open Port Scanner tool — its purpose, architecture, feature set, and technical implementation. It is intended for developers, reviewers, and instructors who want to understand how the project works without reading the source code.

1. Project Overview

The Open Port Scanner is a command-line Python tool designed to scan one or more networked devices for open TCP ports. For each open port it discovers, the tool identifies the service running on that port, detects the underlying protocol, and attempts to grab a banner — the first response the service sends — which often reveals the software name and version. All results are automatically saved to a timestamped text report.

The tool is built entirely on Python's standard library (`socket`, `concurrent.futures`, `datetime`, `os`) with no external dependencies, making it portable and easy to run on any system with Python 3 installed.

2. Features

- **Single Host Scan** — scan one specific IP address.
- **Network Range Scan** — scan all 254 hosts on a /24 subnet.
- **Threaded scanning** using `ThreadPoolExecutor` (100 concurrent workers) for speed.
- **Service name lookup** via `socket.getservbyport()` for well-known ports.
- **Two-stage Protocol Detection:** banner-based matching first, port-number fallback.
- **Banner Grabbing:** connects to each open port and reads the first response line.
- **Input validation:** enforces port range 1–65535 and $\text{start} \leq \text{end}$.
- **Auto-generated** timestamped `.txt` report saved alongside the script.

3. How It Works

3.1 TCP Connection Test

For each port in the requested range, the scanner opens a TCP socket and calls `connect_ex()`. This returns 0 if the connection succeeded (port is open) or a non-zero error code if the port is closed or filtered. The socket timeout is set to 0.5 seconds — short enough to keep scans fast, long enough to avoid false negatives on local and LAN targets.

3.2 Threaded Scanning

All port checks run in parallel through a `ThreadPoolExecutor` pool capped at 100 workers. Each port is submitted as an independent task; futures are collected as they complete. The final list is sorted by port number before display. This approach avoids the overhead of spawning one OS thread per port (which caused dropped results on large ranges) while still achieving near-simultaneous scanning.

3.3 Service Name Lookup

Once a port is confirmed open, `socket.getservbyport()` maps the port number to its IANA-assigned service name (e.g. 80 → http, 22 → ssh). For ports outside the standard registry the field shows 'unknown'.

3.4 Protocol Detection

Protocol detection runs in two stages:

- **Banner-based (primary):** the raw response from the port is matched against a list of known protocol signatures. For example, a response beginning with SSH- is classified as SSH; HTTP/ as HTTP; +OK as POP3.
- **Port-based fallback:** if the banner is empty or does not match any signature, a lookup table of 50+ well-known port-to-protocol mappings is used (e.g. port 3306 → MySQL, port 3389 → RDP, port 5432 → PostgreSQL).

3.5 Banner Grabbing

After confirming a port is open, the scanner reconnects with a 1.5-second timeout and sends a protocol-appropriate probe. Common probes include an HTTP GET request for ports 80/443/8080, a PING command for Redis (port 6379), and an empty read for services like SSH and FTP that send a greeting unprompted. The first line of the response (up to 100 characters) is stored as the banner.

4. Code Structure

The project is a single file: `PortScan.py`. The table below summarises every function.

Function	Responsibility
<code>print_banner()</code>	Prints the ASCII title banner to the terminal.
<code>get_service_name()</code>	Maps a port number to its IANA service name via socket.
<code>detect_protocol()</code>	Two-stage protocol detection: banner match then port lookup.
<code>grab_banner()</code>	Connects to open port, sends probe, returns first response line.
<code>scan_port()</code>	Checks a single port; returns a result dict if open, else None.
<code>tcp_scan()</code>	Runs <code>scan_port()</code> across a port range using <code>ThreadPoolExecutor</code> .
<code>print_results()</code>	Formats and prints the open port table to the terminal.
<code>scanHost()</code>	Orchestrates a full scan of one IP address.
<code>scanRange()</code>	Iterates over all 254 hosts in a /24 subnet.
<code>generate_report()</code>	Writes a timestamped .txt scan report to the script directory.
<code>main()</code>	Entry point: collects user input, dispatches scan, saves report.

5. Data Flow

The sequence below traces the program from start to report output.

```
User runs:  python PortScan.py
            |
            v
        main()                                <- collects: scan type, target IP/network, start port, end port
            |
            v
    scanHost() / scanRange()
            |
            v
    tcp_scan()                                <- ThreadPoolExecutor (100 workers)
            |
    +--- scan_port(ip, port)                  [runs in parallel for each port]
            |
            |      +-- connect_ex()           --> port open?
            |      +-- get_service_name()     --> e.g. 'http'
            |      +-- grab_banner()          --> e.g. 'HTTP/1.1 200 OK'
            |      +-- detect_protocol()      --> e.g. 'HTTP'
            |
            v
    returns { port, service, protocol, banner }
            |
            v
    print_results()    --> formatted table in terminal
    generate_report()  --> scan_report_YYYYMMDD_HHMMSS.txt
```

6. Supported Protocol Signatures

Banner-based detection covers the protocols listed below. If none match, the port-number fallback table covers 50+ additional protocols.

Protocol	Detection Method
SSH	Banner contains 'SSH-'
FTP	Banner contains '220', 'FTP', 'ProFTPD', 'vsftpd', etc.
SMTP	Banner contains 'ESMTP', 'Postfix', 'Sendmail', 'Exchange'
POP3	Banner starts with '+OK'
IMAP	Banner contains '* OK' or 'IMAP'
HTTP	Banner contains 'HTTP/', '200 OK', '301 Moved', etc.
HTTPS	Banner contains 'SSL', 'TLS', or TLS handshake bytes
MySQL	Banner contains 'mysql', 'MariaDB'
Redis	Banner contains '-ERR', '+PONG', or '*1'
MongoDB	Banner contains 'MongoDB' or 'ismaster'
VNC	Banner starts with 'RFB'
Telnet	Banner contains Telnet IAC negotiation bytes
SMB	Banner contains SMB protocol magic bytes
RDP	Banner contains RDP negotiation bytes

7. Sample Terminal Output

```
=====
      Open Port Scanner
=====

OPEN PORT SCANNER
FOUR CHAMPS
=====

Select scan type:
  1. Single Host
  2. Network Range (/24)
Choice (1/2): 1

IP ADDRESS: 192.168.0.113
START PORT: 1
END PORT: 1000

[*] Starting TCP port scan on host 192.168.0.113 (threaded)

Open Ports on 192.168.0.113:
-----
Port    Protocol  Service      Banner
```

```
-----  
80      HTTP      http          HTTP/1.1 200 OK  
135     MSRPC     epmap  
443     HTTPS     https  
445     SMB       microsoft-ds  
-----
```

[+] TCP scan complete

[+] Report saved to: C:\...\scan_report_20260623_143012.txt

8. How to Run

Requirements

- Python 3.x (no third-party packages needed)
- Windows, macOS, or Linux

Command

```
python PortScan.py
```

Steps

1. Choose scan type: **1** for Single Host, **2** for Network Range.
2. Enter the target IP (e.g. 192.168.0.113) or network prefix (e.g. 192.168.0).
3. Enter start and end port numbers (1–65535).
4. Wait for the scan to complete — results print to the terminal.
5. A .txt report is automatically saved in the same folder as the script.

9. Limitations & Possible Improvements

Limitation	Possible Improvement
TCP only — UDP ports not scanned	Add UDP scan mode using SOCK_DGRAM
No progress indicator during scan	Add a live progress bar (e.g. tqdm)
Banner grabbing is sequential per port	Parallelise banner grabbing too
Report is plain text only	Add HTML / PDF report output
No OS fingerprinting	Use TTL and TCP window size heuristics
Network scan is slow (254 hosts serial)	Parallelise host scanning as well
No HTTPS/TLS banner decoding	Use ssl.wrap_socket for TLS handshake

Appendix A — Example Scan Report

Below is an actual report produced by the tool, captured during a single-host scan of 192.168.0.112 across ports 1–10000. The full scan completed in 51.72 seconds and discovered 12 open TCP ports, including FTP, NetBIOS, SMB, and two VMware-related services.

```
=====
                        OPEN PORT SCANNER - SCAN REPORT
=====
Date/Time      : 2026-06-23 21:16:38
Scan Type      : Host Scan
Target         : 192.168.0.112
Port Range     : 1 - 10000
Duration       : 51.72 seconds
=====

Open Ports on 192.168.0.112:

Port    Protocol  Service          Banner
-----
81       unknown    hosts2-ns        N/A
82       unknown    unknown          N/A
99       FTP         unknown          220 Microsoft FTP Service
135      MSRPC       epmap            N/A
139      NetBIOS     netbios-ssn      N/A
445      SMB         microsoft-ds     N/A
902      FTP         unknown          220 VMware Authentication Daemon
                          Version 1.10: SSL Required,
                          ServerDaemonProtocol:SOAP,
                          MKSDisplayPr...
912      FTP         unknown          220 VMware Authentication Daemon
                          Version 1.0,
                          ServerDaemonProtocol:SOAP,
                          MKSDisplayProtocol:VNC
2869     unknown    icslap           N/A
5040     unknown    unknown          N/A
7070     unknown    unknown          N/A
7680     unknown    ms-do            N/A

=====
                        END OF REPORT
=====
```

Observations: ports 902 and 912 expose VMware Authentication Daemon banners that, while not strictly FTP, begin with the FTP-style 220 greeting code — which is why the two-stage detector classifies them as FTP under banner matching. Ports 81, 82, 2869, 5040, 7070, and 7680 returned no banner at all and fell through to the port-based fallback table.